

WEEK 3 SECURITY REPORT

Advanced Security Testing & Final Reporting
NodeGoat Application

Student Name	Fatima Haq
Course	Cybersecurity
Project Title	Securing the NodeGoat Application
Report Week	Week 3 — Advanced Security & Final Reporting
Date	June 26, 2026
Tools Used	Nmap, OWASP ZAP, Winston, Browser DevTools

CONFIDENTIAL – For Educational / Internship Use Only

1. Objective

The objective of Week 3 was to perform advanced security testing, implement application logging, and create a security checklist to improve the overall security posture of the NodeGoat application. This week builds on the vulnerability assessment conducted in Week 1 and the security controls implemented in Week 2.

2. Basic Penetration Testing

Basic penetration testing was performed using Nmap and browser-based testing techniques to identify vulnerabilities and security weaknesses in the locally running NodeGoat application.

2.1 Nmap Port Scanning

Nmap was used to identify open ports and running services on the local machine.

Commands Used

```
"C:\Program Files (x86)\Nmap\nmap.exe" -p 4000 localhost
"C:\Program Files (x86)\Nmap\nmap.exe" -sV localhost
```

Note: -p 4000 scans only port 4000 specifically. -sV performs a service/version detection scan across all default ports.

Scan Results

Port / Protocol	State	Service	Security Note
4000 / TCP	OPEN	NodeGoat Application	Expected (dev only — not for production)
135 / TCP	OPEN	Microsoft RPC	Must not be exposed to public networks
445 / TCP	OPEN	Microsoft SMB	Must not be exposed to public networks

Security Findings

- Port 4000 was expected to be open because the NodeGoat application was running locally.
- Ports 135 (Microsoft RPC) and 445 (Microsoft SMB) must not be exposed to public networks in production environments — these services are common attack vectors for lateral movement and ransomware.

2.2 Cross-Site Scripting (XSS) Testing

A Stored XSS payload was injected into the Address field of the user profile page:

```
<script>alert('XSS')</script>
```

Result

The payload was successfully stored in the database and executed when the profile page was revisited. A browser popup displaying "XSS" appeared.

Finding: Stored Cross-Site Scripting (Stored XSS) confirmed. Severity: HIGH. This means any user visiting an infected profile page would have the script execute in their browser — enabling session hijacking, cookie theft, and phishing redirects.

Note: This is a different and more serious variant than Reflected XSS — the payload persists in the database and affects every user who views the page, not just the attacker themselves.

2.3 SQL Injection Testing

The following payload was tested on the login page:

```
admin' OR '1'='1
```

Result

The login bypass attempt failed. The application correctly rejected the malicious input.

Result: SQL Injection not exploited. NodeGoat uses MongoDB, which uses a different query format — classic SQL injection payloads do not apply. NoSQL injection should be assessed in a future test cycle.

2.4 Penetration Testing Summary

Test	Technique	Severity	Result
Port Scanning	Nmap -sV	INFO	3 open ports identified
Stored XSS	Manual payload injection	HIGH	Vulnerability confirmed — popup appeared
SQL Injection	Manual payload injection	LOW	Not exploited — login bypass failed

3. Logging Implementation Using Winston

To improve monitoring and incident detection capabilities, the Winston logging library was integrated into the NodeGoat application. Proper logging is a critical security control — without logs, it is impossible to detect attacks, trace incidents, or perform forensic analysis.

3.1 Installation

```
npm install winston
```

3.2 Files Created

- logger.js — Custom Winston logger configuration
- security.log — Output log file for security events

3.3 Implementation

A custom logger was created using Winston. The logger captures application startup events and database connection events and writes them to security.log.

```
const winston = require("winston");

const logger = winston.createLogger({
  level: "info",
```

```

format: winston.format.combine(
  winston.format.timestamp(),
  winston.format.printf(({ timestamp, level, message }) =>
    `${timestamp} [${level.toUpperCase()}] ${message}`)
),
transports: [
  new winston.transports.File({ filename: "security.log" }),
  new winston.transports.Console()
]
});

module.exports = logger;

```

Note: The code above is a representative implementation. The actual logger.js created in the project may vary — include your own file content here if different.

3.4 Events Logged

- Database connection established
- Application startup (port 4000)

3.5 Example Log Output

```

2026-06-15T10:23:41.123Z [INFO] Connected to the database
2026-06-15T10:23:41.456Z [INFO] Application started on port 4000

```

Result: security.log file was successfully generated. Security events are now captured with timestamps, enabling incident detection and audit trail creation.

4. Security Checklist

A SECURITY_CHECKLIST.md file was created to document security best practices and track the implementation status across all three weeks of the internship.

Control Area	Description	Status
Input Validation	Validate all user inputs server-side using the validator library	Implemented (Week 2)
Secure Authentication	Use strong session configuration and token-based auth (JWT)	Implemented (Week 2)
Password Hashing	Store passwords using bcrypt with minimum 10 salt rounds	Implemented (Week 2)
HTTPS	Enable HTTPS in production using TLS certificates	Recommended
Security Headers	Apply Helmet.js to set CSP, X-Frame-Options, HSTS, etc.	Implemented (Week 2)
Session Security	Set httpOnly, secure, sameSite flags; enforce session timeout	Partially Implemented
Logging & Monitoring	Log security events using Winston; monitor security.log regularly	Implemented (Week 3)
Vulnerability Assessment	Run OWASP ZAP and Nmap scans periodically	Completed (Weeks 1 & 3)

5. Results

The following security activities were completed successfully during Week 3:

Activity	Status	Week
Nmap Port Scanning	Completed	Week 3
XSS Testing (Stored XSS)	Completed	Week 3
SQL Injection Testing	Completed	Week 3
Winston Logging Implementation	Completed	Week 3
Security Checklist Preparation	Completed	Week 3

6. Recommendations

- Implement proper output encoding and server-side input sanitization to prevent the confirmed Stored XSS vulnerability.
- Move the NodeGoat application to HTTPS in production using a valid TLS certificate.
- Ensure Windows ports 135 and 445 are blocked at the firewall level in any non-local environment.
- Regularly update all project dependencies to patch known vulnerabilities (run npm audit periodically).
- Monitor security.log frequently and set up alerting for error-level events.
- Conduct periodic penetration testing and vulnerability assessments — at minimum before each release.
- Complete JWT verification middleware (flagged in Week 2) to fully close the authentication gap.
- Assess for NoSQL injection vulnerabilities specific to MongoDB (e.g. \$where, \$regex abuse).

7. Conclusion

Week 3 focused on advanced security testing and security monitoring. Basic penetration testing using Nmap identified three open ports, of which ports 135 and 445 represent a risk if exposed to public networks. Browser-based testing confirmed a Stored Cross-Site Scripting vulnerability in the profile Address field, while SQL Injection was not successfully exploited.

Winston logging was successfully implemented to record security-relevant events, providing the foundation for ongoing monitoring and incident detection. A comprehensive security checklist was prepared covering eight control areas, spanning input validation, authentication, encryption, headers, sessions, and monitoring.

Combined with the findings from Week 1 and the security controls implemented in Week 2, this three-week engagement has significantly improved the security posture of the NodeGoat application. The

Stored XSS vulnerability remains the highest-priority open item and should be remediated before any production deployment.

This report was produced as part of a cybersecurity internship training exercise on a deliberately vulnerable application in a local environment. No real or production systems were tested.